

ELECTRICAL AND INFORMATION ENGINEERING

University of the Witwatersrand, Johannesburg

Software Development II

Lab and Project Submissions on GitHub

1 Introduction

The following document describes a number of different aspects related to the use of Git and GitHub in the course. Section 2 shows how to make a correct pull request on GitHub in order to submit the labs. Section 3 takes the form of an FAQ with questions and solutions to common Git/GitHub issues that may occur when doing the labs. Section 4 describes the requirements with respect to project submissions and the various Git workflows that can be used.

2 Lab Submissions — Making a Pull Request on GitHub

Before you can submit your lab via a pull request on GitHub, you need to have combined and pushed all of your solutions (on the *solutions* branch) to your lab's repo on GitHub. Navigate to your repository on GitHub in your web browser, and click on the *Pull requests* tab, followed by the green *New pull request* button. Make sure that *main/master* is set as the base, and *solutions* is set as the branch being compared, and click *Create pull request*. Title your pull request according to the following convention (and do not provide a description):

```
2025-lab-<X>-session-<Y>
```

where you replace `<X>` with the lab number and `<Y>` with the number of the session to which you have been allocated.

For example, if you are submitting lab 1 and the review session to which you have been allocated is number 2, then you would name your pull request:

```
2025-lab-1-session-2
```

Note that:

- Your pull request must be named *exactly* according to the naming scheme given above.
- Your pull request must be *open*. Do not merge the pull request or close it.

We will search for your lab submission by clicking on search bar at the top of the GitHub page and typing the search query in the search box that appears (see Figure 1). The search query will have the following general form:

```
"<pull request name>" is:pr is:open org:witseie-elen3009
updated:< <submission deadline date and time (ISO8601 format)>
```

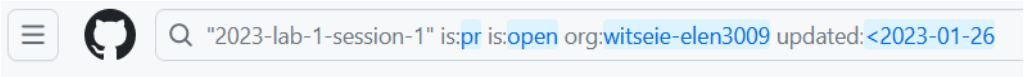


Figure 1: Searching for a pull request

The `updated` part will contain the date and time of the submission deadline for the lab. For a midnight deadline, no time needs to be specified. For example, for lab 1 in session 1, due at midnight on the 25th of January 2023, it will be:

```
"2023-lab-1-session-1" is:pr is:open org:witseie-elen3009 updated:<2023-01-26
```

Search for your own pull request, as shown above, to be sure that it can be found. If it cannot be found, you will receive a 0 for the lab and it will not be marked. If your search does not find your pull request then you have either named it incorrectly, pushed commits to your solutions branch after the deadline, have not opened a pull request, or closed, or merged the pull request. Refer to Section 3.5 to see how to fix these issues.

3 Correcting Lab Submission Mistakes

This section explains how to correct common errors. The following subsections describe the different approaches to take depending on the point at which you discover the mistake. They are organized from discovering the mistake early (when you are still on the `section` branch) to discovering the mistake very late (after it has already been merged and pushed to GitHub). In cases where the problematic `section` branch has already been merged into `solutions`, you will first need to deal with the merge, and then only fix the `section` branch. The instructions given below will guide you.

A general word of warning: *be wary of advice which suggests overwriting your commit history, by force pushing, if you have already shared your code on GitHub*. If you rewrite history which collaborators have based their work off of, you end up creating a very confusing situation which is difficult to recover from, and you run the risk of losing work. None of the approaches suggested here involve force pushing.

3.1 Branch backup

Before making any changes to your commit history it is recommended that you "backup" the branch that you are trying to fix. This is easily done by switching to the problematic branch and creating a backup branch from the tip:

```
git branch <problematic-branch>-backup
```

Now, if anything goes wrong you will still have your original commit history available in the backup branch.

3.2 How do I fix a mistake on a `section` branch which has not yet been merged into `solutions`?

Two approaches to fixing the issue/s are:

1. Create a new branch which branches off from the `section` branch just before the issue occurs, and make correct commits on this new branch (Section 3.2.1). Often this is the easier approach if you are not that familiar with Git.

2. Fix the commits directly on the existing `section` branch (Section 3.2.2).

3.2.1 Create a new `section` branch with correct commits

Identify the commit hash of the commit that is *just before* the first incorrect commit. You can do this by either viewing the `section` branch's commit history (`git log --oneline`) or by using the Git Graph extension. You can then copy the commit hash. Now switch to this commit, and create a new branch from it, by using:

```
git switch -c <new-section-branch> <commit-hash>
```

The name of `new-section-branch` should be a version 2 of the branch that has the issue. For example, if you have made a mistake on the `section-3` branch you could name the new section branch `section-3-v2`.

You can now make the correct commits on this new version of the branch, and then merge this version into `solutions`. In this way the incorrect commit/s will simply not be merged into the `solutions` branch. You can still access these commits (if you need to copy code from them, for example) by switching to the original version of the `section` branch.

3.2.2 Fix commits on the existing `section` branch

Here are a grab bag of different techniques for directly fixing commits. Once again, they are organized from techniques which can be used when the mistake is discovered early (before it is committed) to those which can used when the mistake is discovered later (after it has already been committed).

Be aware, that techniques which modify individual commits or the commit history, *rewrite* the history. You should not do this if a branch is shared with collaborators. Here, however, this is not a problem as the `section` branches are private.

3.2.2.1 How do I throw away changes that I have made, but not yet committed, and restore the file/s from the Git repo?

You can undo *unstaged* changes as follows:

```
git restore . // undoes changes to all files in the current directory
git restore <path-to-file> // undoes changes to the specified file
```

To undo both *staged and unstaged* changes, use:

```
git restore -SW . // undoes changes to all files in the current directory
git restore -SW <path-to-file> // undoes changes to a particular file
```

3.2.2.2 How do I fix the most recent commit on a `section` branch?

The `git commit --amend` command allows you to modify the most recent commit on a branch. This can be useful for correcting mistakes in the commit message or adding files/changes that were forgotten in the original commit.

If you want to edit the last commit message, run:

```
git commit --amend
```

This opens your default text editor, allowing you to modify the message. Save and exit to apply the changes.

If you forgot to include files, or have modified included files, first stage the changes:

```
git add <file-name>
```

Then run:

```
git commit --amend
```

3.2.2.3 How do I undo commits on a `section` branch?

You might want to undo one or more recent commits but keep their changes around, so you can tweak and re-commit. To do this type:

```
git reset HEAD~n // n refers to how many commits you want to undo,
                 // eg: HEAD~1 undoes the last commit
```

This command undoes the last *n* commits and unstages all the changes, allowing you to modify them before committing again.

3.2.2.4 How do I change any commit on my `section` branch? (Advanced)

Git's interactive `rebase` command is sometimes referred to as the "Swiss Army knife" of Git commands. It is a powerful command which allows you to reword, squash, drop, reorder and amend commits. You are not expected to know how to use `rebase` in this course. However, if you wish to try it out to fix a branch issue, read on.

First switch to the problematic `section` branch and then type the following:

```
git rebase -i --keep-base solutions // rebase on to the base commit from
    solutions that the branch was created from
```

Git will open a generated `git-rebase-todo` file in your editor. The top lines correspond to the actions to take during rebase. The default list of actions picks all of the existing commits on the branch, in their current order. Note that the order is *earliest-first*, unlike git log's default of latest-first.

If necessary, you can edit the list:

- reorder commits by reordering the lines,
- change actions by replacing "pick" with another action letter or name, and/or
- unselect/drop commits by deleting their corresponding lines.

After you save and close the file, and Git executes the rebase as scheduled.

Watch this video on rewriting Git history and this one on squashing commits for more detail.

After you have completed the process, run the `log` command to check that the changes have been made. You will notice that all of the branch commit hashes will be different. This is because commits are immutable (and so cannot be changed) and therefore new commits have to be made.

3.3 How do I fix a mistake on a `section` branch which has already been merged into `solutions` but the `solutions` branch has not been pushed to GitHub?

Fortunately, the mistake has not yet been pushed to GitHub, so all that needs to be done is to reset the local `solutions` branch to the remote version that is on GitHub, thereby undoing the merge. This is done as follows:

```
git fetch // ensure remote tracking branches are up-to-date
git switch solutions
git reset --keep @{upstream} // reset solutions to the remote version, keep
    any uncommitted changes (if there are any)
```

Your existing problematic `section` branch will still exist but it will no longer be merged into `solutions`. Proceed by following the advice given in Section 3.2 for fixing and merging this `section` branch.

3.4 Whoops, how do I fix a mistake on a `section` branch which has been merged into `solutions`, and I've already pushed it to GitHub?

Here, because the problematic branch has already been merged and shared on GitHub we need to create a new version of `solutions` and use this instead, effectively abandoning the old version. Take the following steps to do this:

1. Pull the `solutions` branch to make sure you are up-to-date. View the `solutions` branch commit history using `git log --oneline`, or use the Git Graph extension, to identify the last correct commit (which may be a *merge* commit), which occurs just before the commits belonging to the problematic branch.
2. Copy the commit hash and create a new branch off `solutions` from this commit using

   ```
   git switch -c solutions-v2 <commit-hash>
   ```

3. Now follow the instructions in Section 3.2 to fix the issues on your local `section` branch and then merge it into `solutions-v2`.
4. Push your new `solutions-v2` branch to GitHub. You will need to set the remote upstream branch:

   ```
   git push --set-upstream origin solutions-v2
   ```

   before you can push.
5. If subsequent section branches were merged into `solutions` after the problematic branch then these will now need to be merged into `solutions-v2` as well.
6. Lastly, when submitting the lab you will make a pull request to pull `solutions-v2` into `main`. The original `solutions` branch will just be ignored.

3.5 Fixing Pull Requests

3.5.1 How do I fix an incorrectly named pull request?

Login to GitHub, click on *Pull request* tab, find your pull request and select it. Click the *Edit* button on the right-hand side of the screen to supply a new name.

3.5.2 How do I reopen a closed pull request?

Login to GitHub, click on *Pull request* tab, click *Closed*, and find your pull request. Clicking on it will allow you to reopen it.

3.5.3 I inadvertently merged my pull request, how do I fix this?

After having done this the *main* and *solutions* branches will be identical and contain all the solutions. However, you need to make a pull request from *solutions* into a `main` branch which only contains the initial commit. Therefore, you will need to create a new `main` branch from the initial commit, called *main-2*. Push this up to GitHub and make a new pull request to pull *solutions* into *main-2*.

4 Project Submissions

The use of Git and GitHub for project submissions is different to that for the laboratories. For project submissions, you are required to:

- conform to a specific directory structure for both the project code and the binaries, game resources, and documentation,
- make commits to *main/master* or merge commits into *main/master* prior to each release, and
- tag and publish releases from your *main/master* branch.

These requirements are discussed in the project brief.

There is no obligation to name commits in a specific way, use specific branches (other than *main/master*), or follow a specific workflow, although some workflows are suggested in Section 4.1.

More detail on Git tags is given in Section 4.2. If you need to delete a release, refer to this section.

4.1 Git Workflows for the Project

In order to *work simultaneously* on different parts of the code base you need to give some thought as to how you will collaborate. Three different workflows with progressive complexity are discussed below. Whichever workflow your group decides to use you should follow these *three key principles*:

1. Communicate with your partner regularly so that it is clear who is working on which feature. To work effectively as a team, group members should not be working on the same feature at the same time, that is, touching the same sections of the code base. You want to limit the likelihood of merge conflicts.
2. Never push broken code (code that does not compile) or failing tests to a shared branch. Sharing broken code means that not only is your own code broken, but you will break your partner's code when they update their version of the branch.
3. Share your code frequently. After every coding session (or perhaps even more often) you should be sharing your changes by merging local work into shared branches and both *pulling and pushing* these branches. The reason for doing this is to minimize the number of merge conflicts that can occur at any one time. If you work for an

extended period of time in isolation, then when the time comes to share your code, you may have to deal with a complicated and difficult merge.

4.1.1 Single Main Branch

The simplest workflow is that you do not make use of branching at all. Each group member works off the shared *main* branch. Members pull and push from *main* periodically to share their code. This workflow illustrated in Figure 2. Although this workflow is really simple, it has disadvantages in that you cannot work on any particular feature or bug-fix in isolation because everything has to happen on *main*. You should be sharing your code often which means that if you use this workflow you will likely be pushing incomplete features to *main* which is not that desirable.

4.1.2 Local Feature Branches Merging Into Main

In this workflow you use local branches to isolate feature development, and *main* is the only shared branch. Merges take place locally, so this has the disadvantage that you cannot perform code reviews. Personally, this is the workflow that I would recommend as it is lightweight, and you are a very small team. The steps in this workflow are as follows:

- The shared *main* branch on GitHub contains the latest, working version of your game. You need to have pulled down *main* so that you are up-to-date. To work on something new, create a local, descriptively-named, branch off *main* (i.e. *feature-x*).
- Checkout this branch and make regular commits to it as you work on the feature.
- When you have completed the feature, or your local branch is ready to be integrated into *main*, pull down (update) *main*, and then merge in your *feature* branch. The reason you need to pull down *main* first is because your partner may have pushed new changes to the remote *main* after you created your feature branch. You need to integrate your changes, on your machine, with the latest version of the code. You may have to resolve merge conflicts if your partner has made a change to *main* which conflicts with changes you have made on your feature branch.
- After resolving all merge conflicts with a merge commit (if required) and ensuring that the *main* branch compiles and runs without errors, push it to GitHub so that your updates become available to your partner.

4.1.3 Shared Feature Branches

This workflow is also known as the "GitHub Flow". Note, you will not have a Deploy step as you are developing desktop-based software.

If you make use of this workflow, you will have the opportunity to review each other's code before merging it into *main*. Performing code reviews has been shown to improve the quality of the code produced.

4.2 Git Tags

Refer to the basics of git tagging for more detailed information.

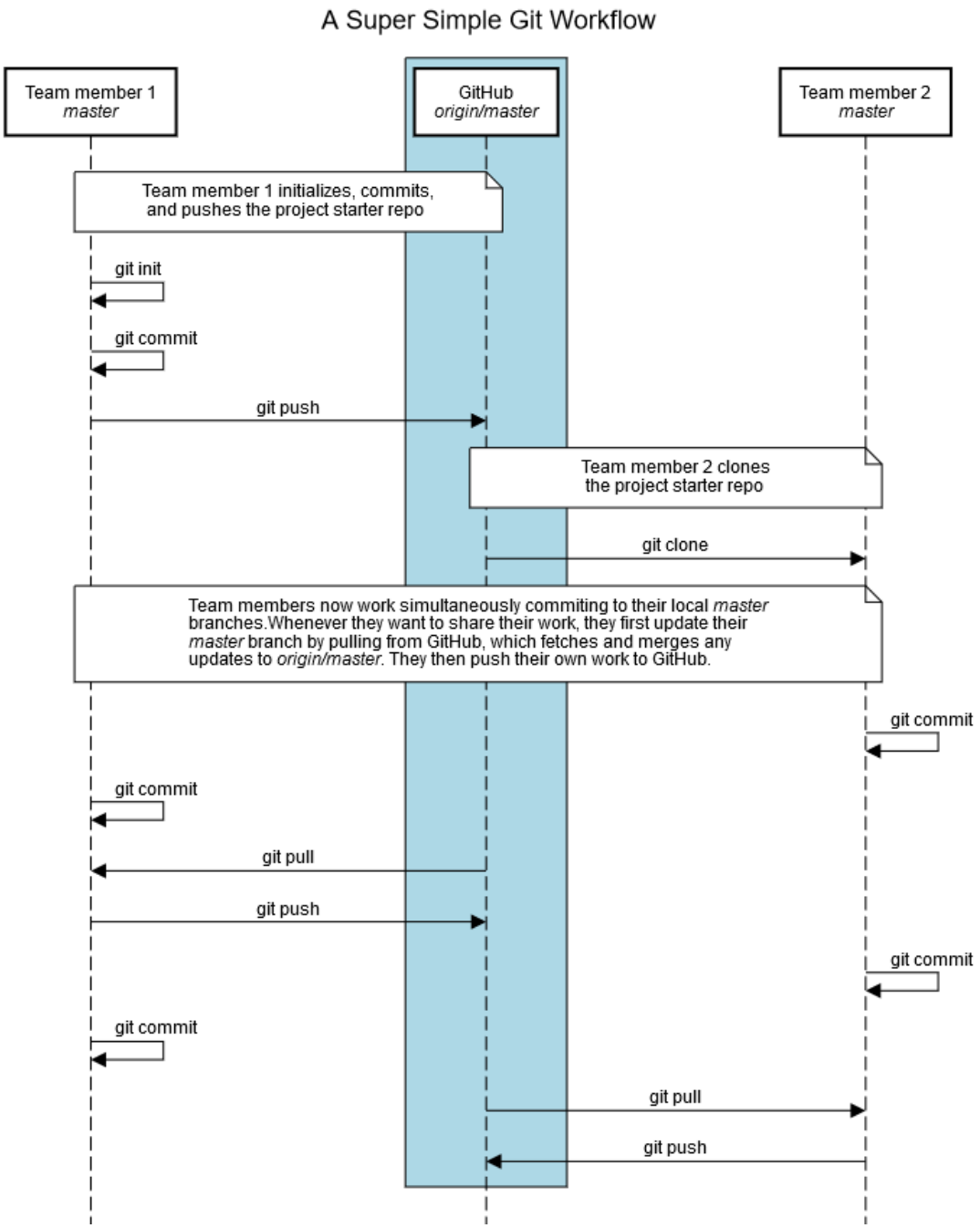


Figure 2: Single *main* branch workflow

4.2.1 How do I see the tags that I have created?

To list all the tags in your local repo, simply type:

```
git tag
```

4.2.2 How do I delete a tag/release?

Assuming you want to remove the `v2.0` tag from your local repo, you would type:

```
git tag -d v2.0
```

If you have already pushed the tag to GitHub, you can remove it from the remote repo by typing the following command:

```
git push -d origin v2.0
```

You then also need to delete the release on GitHub. Go to your repo on GitHub, click on *Releases* in the right-hand side panel. Click on the release that you want to delete and then click on *Delete* on the right-hand side of the page.



Source: http://phdcomics.com/comics/archive.php?comicid=1531

Copyright SP Levitt, Wits School of Electrical and Information Engineering.